

Process API Lezione 22-10-2024

Per creare un file.c nano hello.c

Per salvare ctrl + o + invio

Per chiudere ctrl + x

Per compilarlo gcc -o hello hello.c

Per eseguirlo ./hello

Per spostare un file docker cp sublime.c os2425:/home/corso

Copio la cartella CPU API che si trova in ostep code dato che quella parte quando viene aggiornata l'immagine viene cancellata cp -r cpu-api/ /home/corso

Analisi di p1.c

Fork: Contiene una chiamata di sistema, sto facendo un salto nel kernel, fork sdoppia il processo e duplica la memoria del processo che lo chiama, quindi avremo due processi identici, il processo che ha chiamato la fork lo chiamo processo padre che come valore della fork restituisce il pid del processo figlio, mentre per il processo figlio il valore ritornato sarà 0.

Quindi l'esecuzione del programma dimostra come il processo che ha chiamato la fork imbocca l'ultimo ramo avendo valore != 0, il processo figlio imbocca il ramo con pid=0, mentre se qualcosa fosse andato storto sarebbero andati al primo ramo.

Manuale

Comando man + nomeChiamataDiSistema, per scoprirne la sintassi

Nel caso della fork ci dice cosa fa ci dice che ci restituisce un intero (pid_t). Inoltre specifica che i due processi hanno due spazi di memoria differenti

Wait: Vogliamo leggere i prototipi restituiti da man, pid_t è sempre il valore di ritorno ed è un intero, successivamente ci viene scritto il nome della funzione wait con dentro i parametri da passare, in particolare c'è un puntatore nullable. La descrizione ci dice che le chiamate di sistema servono per aspettare il cambiamento di stato dei processi e ottenere informazioni sul processo figlio e sul cambiamento di stato.

Analisi di p2.c

Rispetto a prima quello che va a fare è una wait di null e quindi ci aspettiamo che nel caso in cui la wait abbia successo ci aspettiamo che la variabile che lo contiene abbia il pid del processo che ha cambiato il suo stato per primo.

Scriviamo da 0 un programma che ci consenta di scrivere dentro un file

O_CREAT crei il file in caso non lo trovi

O_trunc se il file già esiste sarà troncato a 0.

O_WRONLY siamo interessati soltanto a scrivere il file

Attribuisco al file i permessi S_IRWXU il possessore del file può leggerlo, scriverlo e eseguirlo

```

#include <stdio.h>

#include <fcntl.h>

#include <string.h>

#include <unistd.h>

int main(){

    char messaggio[]="ciao";

    int fd=open("pippo.txt", O_CREAT | O_TRUNC | O_WRONLY, S_IRWXU);

    write(fd, messaggio, strlen(messaggio));

    close(fd);

}

```

28-10-2024 (ci sono le slide)

Riprendiamo l'analisi di scrittura_file.c, non capiamo perché non riusciamo a leggere il contenuto di pippo.txt, per farlo dobbiamo utilizzare la chiamata di sistema read(), questa prende come primo parametro un file descriptor, prende un buffer che serve a immettere l'informazione che stiamo leggendo nella memoria.

```

#include <stdio.h>

#include <fcntl.h>

#include <unistd.h>

int main(){

    int fd= open("pippo.txt", O_RDONLY, S_IRWXU);

    int buffer=0;

    read(fd, &buffer, sizeof(int));

    printf("Buffer: %d\n", buffer);

    close(fd);

}

```

Un'ultima cosa da vedere, è la chiamata che ci consente di eseguire altre applicazioni d'al programma che stiamo utilizzando. La chiamata EXEC. Sono presenti varie chiamate, le quali hanno sintassi diverse. Ad esempio execv che ha come primo argomento una stringa che fa riferimento al percorso dell'eseguibile che vogliamo andare a lanciare.

```

GNU nano 7.2      lettura_file.c      S #include <stdio.h>

#include <fcntl.h>

#include <unistd.h>

```

```
#include <string.h>

int main(){

    int fd= open("pippo.txt", O_RDONLY, S_IRWXU);

    int buffer=0;

    read(fd, &buffer, sizeof(int));

    printf("Buffer: %d\n", buffer);

    close(fd);

    char *myarg[2]; //La posizione 0 fa riferimento al percorso dell'eseguibile, seguito dagli argomenti
    che vogliamo passare all'eseguibile, l'ultimo elemento deve essere necessariamente null

    myarg[0]= strdup("ls"); //vado a mettere un puntatore all'interno di myarg 0 di qualcosa che sta all
    interno dell'heap          myarg[1]= NULL;

    execvp(myarg[0], myarg);

    printf("CIAO\n"); //questo ciao non viene letto perch  la mia esecuzione si ferma a execvp

}
```